6
Hashing

Uma ***função de hash*** é uma função $h : U \to N$ que transforma objetos de um conjunto *muito grande* U, o ***universo*** (números, cadeias de caracteres, arquivos, grafos), em valores de um conjunto bem menor N, em geral $N = \{0, 1, \ldots, n-1\}$ ou o conjunto das cadeias de k bits. Como $|U|$ é muito maior que $|N|$, a função *comprime* a informação: o valor $h(x)$ é uma espécie de "resumo" de x, que ocupa pouco espaço e pode ser comparado e usado como índice com custo constante. O preço dessa compressão é que elementos distintos podem receber o mesmo valor, o que chamamos de *colisão*, e que, pelo princípio da casa dos pombos, não pode ser evitado.

O que torna uma função de *hash* útil não é, portanto, ser injetora, mas ter algumas propriedades que dependem do uso:

- ser ***fácil de computar***: calcular $h(x)$ deve custar pouco tempo, idealmente tempo constante;
- ter ***descrição curta***: poucos bits devem bastar para armazenar a função;
- ***espalhar bem*** os elementos que de fato aparecem: dado um conjunto $S \subseteq U$, os valores $h(x)$, $x \in S$, devem se distribuir de modo aproximadamente uniforme em N, de modo que as colisões sejam poucas.

A terceira propriedade é a mais delicada. Para qualquer função fixa existem conjuntos S em que todos os elementos colidem, de modo que nenhuma h determinística espalha bem *todos* os conjuntos. A saída, que é o fio condutor deste capítulo, é usar ***aleatoriedade***: sortear h de uma família $\mathcal{H}$ de funções e analisar o comportamento esperado, ou com alta probabilidade, para qualquer S fixo. Uma função totalmente aleatória espalha bem, mas não tem descrição curta; o problema central é encontrar famílias pequenas, de funções fáceis de computar, que preservem as propriedades estatísticas de que precisamos. É nesse ponto que entram as famílias universais.

As funções de *hash* aparecem em muitos contextos: como estrutura de dados para operações de dicionário, na verificação rápida de igualdade e de integridade de dados ("impressões digitais"), na teoria da complexidade e na desaleatorização de algoritmos, e na criptografia, onde se exige de h propriedades bem mais fortes, como a dificuldade de encontrar colisões. Nosso foco são as propriedades *probabilísticas* de famílias de funções.

6.1 Tabelas de espalhamento

Uma ***tabela de espalhamento***, ou ***tabela hashing***, é uma solução bastante conhecida e estudada para a representação computacional de um conjunto, seja ele estático ou dinâmico. Um modo de implementar uma tabela de espalhamento é usar um vetor H cujas entradas são listas ligadas e o acesso à tabela é feito por meio de uma ***função de hash*** $h : U \to \{0, 1, \ldots, n-1\}$. Dado $x \in U$, a inserção, remoção ou busca de x em H é realizada na lista ligada $H[h(x)]$ (veja uma ilustração na Figura 6.1).

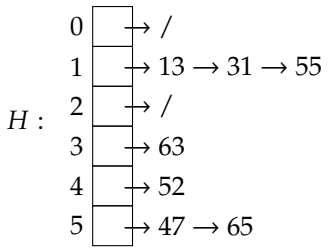


Figura 6.1: tabela de espalhamento com os elementos $\{13, 31, 47, 52, 55, 63, 65\}$ distribuídos de acordo com a função $h(x) = x \bmod 6$.

As operações de busca, inserção e remoção de um elemento numa tabela H que representa um conjunto dinâmico $S \subset U$ com função de *hash* $h : U \to N$, para $N = \{0, 1, \ldots, n-1\}$, são descritas a seguir.

busca: dado $x \in U$, uma operação de busca por x em S responde à pergunta "$x \in S$?" e ainda, no caso positivo, retorna um apontador para x. Numa tabela de espalhamento isso é realizado por uma busca sequencial na lista ligada $H[h(x)]$. Uma busca por x em H é dita *com sucesso* caso $x \in H$, senão é dita *sem sucesso*. O custo (ou tempo) de pior caso de uma busca é proporcional ao número de elementos na maior lista;

inserção: dado $x \in U$, uma operação de inserção acrescenta na estrutura que representa S o elemento x, caso esse ainda não faça parte de S. A inserção propriamente dita numa tabela de espalhamento é simplesmente inserir o elemento x no início da lista ligada $H[h(x)]$ e o custo dessa operação é constante;

remoção: dado $x \in U$, a remoção de x retira esse elemento da estrutura

que representa S, caso $x \in S$. A remoção propriamente dita numa tabela de espalhamento, dada a posição de x na lista ligada através de um apontador, é a remoção do elemento de uma lista ligada e o custo de pior caso dessa operação também é constante.

Para a análise do desempenho das operações de dicionário nessa estrutura de dados, o tempo de busca de um item é importante e, no pior caso, é o tempo de busca na lista $H[i]$ com o maior número de elementos. No emprego de tabelas de espalhamento, usualmente, temos a seguinte situação: o ***conjunto universo*** U é muito grande, o que significa que o custo de uma representação do conjunto S usando, por exemplo, um vetor de bits de tamanho $|U|$ é proibitivo. Se fosse viável, teríamos uma estrutura com tempo de busca $O(1)$. O conjunto S, de cardinalidade $m := |S|$, é uma fração de U e o caso interessante é quando $m = \Omega(n)$. É desejável que os elementos de S fiquem bem espalhados na tabela, pois ***colisões*** (elementos distintos mapeados para a mesma lista) afetam o desempenho das operações. Como U tem muito mais que n elementos não há como evitar colisões.

Das funções de *hash*, além do bom espalhamento, queremos que seja fácil de computar e que a representação seja sucinta, isto é, relativamente poucos bits sejam necessários para armazenar a função. Esse tipo de função, além de uma grande ferramenta prática como descrevemos abaixo, é bastante útil em Complexidade Computacional e em Criptografia.

Para qualquer h, existe um conjunto S em que todos os elementos colidem, basta escolher m elementos de U na mesma pré-imagem, o que sempre pode ser feito pois tem tamanho pelo menos $|U|/n$ e U é assumido grande. Por isso o desempenho precisa ser analisado em média, e a aleatoriedade entra de forma natural.

Hashing com funções aleatórias Uma função h escolhida uniformemente no conjunto N^U de todas as $|N|^{|U|}$ funções de U em N produz, quando $n = \Theta(m)$, listas de tamanho no máximo $O(\log m)$ com alta probabilidade. Isso garante, com alta probabilidade, um tempo de busca equivalente ao das árvores de buscas, com a vantagem de manutenção mais simples. Além disso, o tempo médio de busca é $\approx m/n = O(1)$, quando $n = \Theta(m)$.

Nesse modelo probabilístico, a escolha de uma função aleatória equivale a escolher, independentemente para cada elemento $x \in U$, uma posição $h(x)$ uniformemente em N.

Um problema nessa estratégia é que tal função não tem uma representação sucinta. Algumas delas requerem pelo menos $|U| \log(n)$ bits para serem representadas. Por ora analisaremos o caso de uma função aleatória para um conjunto S fixo, porém arbitrário. Nesse caso o tamanho de uma lista é uma variável aleatória.

Vamos considerar o modelo probabilístico $(N^U, \mathbb{P})$ com $\mathbb{P}(\, h \,) = 1/|N^U|$ para toda função $h \in N^U$. Notemos que vale

$$\mathbb{P}[\, h(x) = i \,] = \frac{1}{|N|} \tag{6.1}$$

para todo $x \in U$ e todo $i \in N$. De fato, sortear uniformemente uma função é equivalente a sortear uniformemente e independentemente uma imagem para cada elemento do domínio.

Fixamos os parâmetros $m = |S|$ e $n = |N|$ e então a fração

$$c = c(n, m) \coloneqq \frac{m}{n} \tag{6.2}$$

é denominada ***carga*** da tabela. A carga da tabela é o valor esperado para a quantidade de elementos de S numa mesma posição da tabela, segundo a medida da equação (6.1). Tomemos a variável aleatória

$$\mathbb{1}_{[h(x)=h(y)]} \colon N^U \to \{\, 0, 1 \,\}$$

indicadora de colisão dos elementos $x, y \in U$ sob a escolha $h \in N^U$. A probabilidade de colisão é

$$\begin{aligned} \mathbb{P}[\, h(x) = h(y) \,] &= \sum_{i \in N} \mathbb{P}[\, h(x) = i \,] \mathbb{P}[\, h(y) = i \,] \qquad (x \neq y) \\ &= n \cdot \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n}. \end{aligned} \tag{6.3}$$

assim, o número de itens na lista $H[h(x)]$ é dado por $\sum_{y \in S} \mathbb{1}_{[h(x)=h(y)]}$ e

$$\mathbb{E}\mathbb{1}_{[h(x)=h(y)]} = \begin{cases} \frac{1}{n}, & \text{se } y \neq x \\ 1, & \text{caso contrário,} \end{cases}$$

portanto o tamanho esperado da lista $H[h(x)]$ é, pela linearidade da esperança

$$\sum_{y \in S} \mathbb{E}\mathbb{1}_{[h(x)=h(y)]} = \begin{cases} \frac{m}{n}, & \text{se } x \notin S, \\ \frac{(m-1)}{n} + 1, & \text{caso contrário.} \end{cases} \tag{6.4}$$

O tempo esperado de uma busca em uma tabela de espalhamento com uma função *hash* h escolhida aleatoriamente em N^U é proporcional à carga c da tabela e, portanto, é constante quando $n = \Theta(m)$.

Proposição 6.1

Sejam $S \subseteq U$, $|S| = m$, e H uma tabela de espalhamento com n posições, usando uma função de *hash* $h \colon \quad U \to N$ escolhida aleatoriamente. Suponha que os elementos de S são inseridos no início da lista ligada correspondente a cada célula de H. Então:

1. o tempo esperado de uma busca *sem sucesso* é $c+1$,
2. o tempo esperado de uma busca *com sucesso* é $1+\frac{c}{2}-\frac{1}{2n}$,

onde $c=m/n$ é a carga média da tabela.

Demonstração. Se $x \notin S$, a busca percorre todos os elementos na lista $H[h(x)]$. Pelo modelo de função aleatória (equação (6.4)), o tamanho esperado dessa lista é $m/n = c$, assim chegamos ao final da lista com $c+1$ comparações em média.

Suponha que $x \in S$ e que foi o i-ésimo elemento inserido. Como inserimos sempre no início da lista, x estará na frente da lista no momento da inserção. Dos $m-i$ elementos restantes que ainda serão inseridos, em média $(m-i)/n$ colidirão com x, de modo que o tempo esperado para achar x nessa lista será $(m-i)/n+1$ comparações. A média sobre todos os possíveis valores de i é

$$\frac{1}{m}\sum_{i=1}^{m}\left(1+\frac{m-i}{n}\right) = 1+\frac{1}{mn}\sum_{i=1}^{m}(m-i) = 1+\frac{m-1}{2n}.$$

Assim, o tempo médio de busca com sucesso é $1+c/2-1/(2n)$. □

Lema 6.1: a maior lista é sublogarítmica

Para $X_i := |\{ y \in S : h(y)=i \}|$, n suficientemente grande e $c \leqslant 1$,

$$\mathbb{P}\left[\max_i X_i \geqslant \frac{8\ln n}{\ln\ln n}\right] \leqslant \frac{1}{n}.$$

Como $n=\Theta(m)$, temos $\ln n=\Theta(\ln m)$, e a maior lista tem tamanho $O(\log m/\log\log m)$ com probabilidade $\geqslant 1-1/n$.

Demonstração. Fixemos $i \in N$ e $k \geqslant 1$. Se $X_i \geqslant k$, existe $T \subseteq S$ com $|T| = k$ e $h(y) = i$ para todo $y \in T$. Pela independência dos valores $h(y)$ e por $\mathbb{P}[\, h(y)=i \,] = 1/n$, cada T tem probabilidade n^{-k}. Usando a subaditividade em T dá

$$\mathbb{P}[\, X_i \geqslant k \,] \leqslant \binom{m}{k}\frac{1}{n^k} \leqslant \frac{m^k}{k!\,n^k} = \frac{c^k}{k!} \leqslant \frac{1}{k!},$$

onde usamos $\binom{m}{k} \leqslant m^k/k!$ e $c = m/n \leqslant 1$. Para limitar $k!$, note que $e^k = \sum_{j\geq 0} k^j/j! \geqslant k^k/k!$, logo $k! \geqslant (k/e)^k$ e

$$\mathbb{P}[\, X_i \geqslant k \,] \leq \left(\frac{e}{k}\right)^k. \tag{6.5}$$

Logo, sobre as n posições

$$\mathbb{P}\left[\max_i X_i \geqslant k\right] \leqslant n\left(\frac{e}{k}\right)^k = n\exp(-k(\ln k-1)).$$

Aqui não precisamos de independência entre as listas, que de fato não são independentes (a soma dos tamanhos é m).

Tome $k_0 = 8\ln n/\ln\ln n$, $k = \lceil k_0 \rceil$ e escreva $y = \ln\ln n$. A função $\phi(t) := t(\ln t - 1)$ é crescente para $t \geqslant 1$, pois $\phi'(t) = \ln t \geqslant 0$. Como $k \geqslant k_0$, vale $\phi(k) \geqslant \phi(k_0)$, e basta estimar $\phi(k_0)$. Temos

$$\ln k_0 = \ln 8 + y - \ln y \geqslant \tfrac{1}{2}y,$$

pois isso equivale a $\ln y \leqslant \frac{1}{2}y + \ln 8$, que vale para todo $y > 0$ (o máximo de $\ln y - \frac{y}{2}$ é $\ln 2 - 1 < 0$). Para n grande o bastante, $y \geq 4$, e então $\ln k_0 - 1 \geqslant \frac{1}{2}y - 1 \geqslant \frac{1}{4}y$. Logo

$$\phi(k_0) = k_0(\ln k_0 - 1) \geqslant \frac{8\ln n}{y} \cdot \frac{y}{4} = 2\ln n,$$

e portanto, como $\phi(k) \geqslant \phi(k_0) \geqslant 2\ln n$,

$$\mathbb{P}\left[\max_i X_i \geqslant k\right] \leqslant n\, e^{-\phi(k)} \leqslant n \cdot e^{-2\ln n} = \frac{1}{n}. \qquad \square$$

Observação 14. Se $n = \Theta(m)$ com $c \leq C$, a cota em (6.5) fica $(Ce/k)^k$. Basta trocar e por Ce e ajustar a constante 8 que o resultado é o mesmo. ◊

Observação 15 (*A cota é justa*). Prova-se, via um argumento de segundo momento sobre o número de listas com $\geqslant k$ elementos, que com alta probabilidade a maior lista tem tamanho $\Omega(\log n/\log\log n)$. Portanto, a maior lista é $\Theta(\log n/\log\log n)$. ◊

6.1.1 *O paradoxo dos aniversários e um limiar para colisões*

Para S fixo e C a quantidade de pares de S que colidem, o número esperado de colisões é

$$\mathbb{E}C = \binom{m}{2}\frac{1}{n} = \frac{m(m-1)}{2n} \tag{6.6}$$

assim, usando (3.5)

$$\mathbb{P}[\,C = 0\,] \geqslant 1 - \mathbb{E}C = 1 - \frac{m(m-1)}{2n} \tag{6.7}$$

e no caso $m = \sqrt{n}$ temos $\mathbb{E}C < 1/2$ e $\mathbb{P}[\,C = 0\,] \geqslant 1/2$.

Para $m \leqslant n$ há $n(n-1)(n-2)\cdots(n-m+1)$ sequências sem repetições em N^m. Se $(h(x_1), h(x_2), \ldots, h(x_m))$ é uma escolha aleatória

em N^m e C é o número de colisões, então nessa escolha

$$\begin{aligned}\mathbb{P}[\,C=0\,] &= \frac{n(n-1)(n-2)\cdots(n-m+1)}{n^m}\\ &= \left(1-\frac{1}{n}\right)\left(1-\frac{2}{n}\right)\cdots\left(1-\frac{m-2}{n}\right)\left(1-\frac{m-1}{n}\right)\\ &< \exp\left(-\frac{1}{n}\right)\exp\left(-\frac{2}{n}\right)\cdots\exp\left(-\frac{m-1}{n}\right)\\ &= \exp\left(-\frac{m(m-1)}{2n}\right)\end{aligned}$$

na segunda linha usamos que $\exp(-x) > 1-x$ (veja (d.1)).

O ***paradoxo dos aniversários*** é o caso $m = 23$ e $n = 365$ na equação acima: $\mathbb{P}[\,C>0\,] = 1-\mathbb{P}[\,C=0\,] = 1-\exp(-506/730) > 0{,}5$ (pois $506/730 \approx 0{,}693151 > \ln 2 \approx 0{,}693147$), ou seja, apenas 23 pessoas são suficientes para que duas delas façam aniversário no mesmo dia com probabilidade maior que $1/2$, supondo que os nascimentos ocorram uniformemente ao longo do ano.

Para todo real $x \in (0, 3/4]$ vale[1] $1-x > \exp(-2x)$.

[1] Seja $f(x) = \ln(1-x)+2x$. Então $f(0)=0$ e $f'(x) = 2-\frac{1}{1-x}$, que é positiva em $(0,1/2)$ e negativa em $(1/2,3/4]$. Logo f cresce e depois decresce, e portanto $f(x) > \min\{f(0), f(3/4)\}$ em $(0,3/4]$. Como $f(3/4) = \frac{3}{2} - \ln 4 > 0$, segue que $f > 0$ em todo o intervalo.

Aplicando essa desigualdade a $x = j/n$, com $1 \leqslant j \leqslant m-1$, o que é permitido quando $m \leqslant (3/4)n$, obtemos

$$\begin{aligned}\mathbb{P}[\,C=0\,] = \prod_{j=1}^{m-1}\left(1-\frac{j}{n}\right) &> \prod_{j=1}^{m-1}\exp\left(-\frac{2j}{n}\right)\\ &= \exp\left(-\frac{m(m-1)}{n}\right).\end{aligned}$$

Em particular, se $m \leqslant \sqrt{n}$, então $m(m-1)/n < 1$ e

$$\mathbb{P}[\,C=0\,] > e^{-1} > 0{,}36.$$

Ou seja, para $m = O(\sqrt{n})$ a probabilidade de inserir os m elementos sem colisão é limitada inferiormente por uma constante positiva.

Por outro lado, a cota superior $\exp\left(-\frac{m(m-1)}{2n}\right)$ obtida acima tende a 0 quando $m = \omega(\sqrt{n})$, e portanto $\sqrt{n}$ é o limiar para o aparecimento de colisões.

Markov × exp Para $m \leqslant \sqrt{n}$, a desigualdade de Markov, usada no início dessa seção, fornece uma cota melhor. Como $\mathbb{E}C = \frac{m(m-1)}{2n} < \frac{1}{2}$, temos

$$\mathbb{P}[\,C=0\,] \geqslant 1-\mathbb{E}C > \frac{1}{2} > e^{-1}.$$

A cota exponencial acima é mais fraca nesse regime porque o fator 2 em $1-x > \exp(-2x)$ foi escolhido para valer em todo o intervalo $(0,3/4]$, e não é justo para x pequeno. Em compensação, ela vale para todo $m \leqslant (3/4)n$, onde a cota de Markov deixa de ser informativa: para $m \geqslant \sqrt{2n}+1$ temos $\mathbb{E}C \geqslant 1$, e a cota $1-\mathbb{E}C$ torna-se trivial.

6.1.2 *Variância do número de colisões e do tamanho de uma lista*

Para uma função $h \in_{\mathrm{R}} N^U$, os valores $h(x)$, com $x \in U$, são independentes e uniformes em N. Em particular, para todos $x \neq y$ em U e todos $i, j \in N$, os indicadores $\mathbb{1}_{[h(x)=i]}$ e $\mathbb{1}_{[h(y)=j]}$ são independentes, isto é,

$$\mathbb{P}(\,[h(x)=i]\cap[h(y)=j]\,) = \frac{|\,\{\,h\in N^U\colon h(x)=i,\ h(y)=j\,\}\,|}{n^{|U|}} = \frac{1}{n^2}$$

e, em particular, $\mathbb{P}[\,h(x)=h(y)\,] = \sum_{i\in N}\mathbb{P}(\,[h(x)=i]\cap[h(y)=i]\,) = 1/n$.

Seja X_i o número de elementos de S que caem na posição i

$$X_i = \sum_{x\in S}\mathbb{1}_{[h(x)=i]}.$$

Sabemos que $\mathbb{E}X_i = c$. Como as variáveis indicadoras são duas a duas independentes,

$$\begin{aligned}\operatorname{Var}(\,X_i\,) = \operatorname{Var}\left(\sum_{x\in S}\mathbb{1}_{[h(x)=i]}\right) &= \sum_{x\in S}\operatorname{Var}\left(\,\mathbb{1}_{[h(x)=i]}\,\right)\\ &= \sum_{x\in S}\tfrac{1}{n}\left(1-\tfrac{1}{n}\right) = m\frac{1}{n}\left(1-\frac{1}{n}\right) = c\left(1-\frac{1}{n}\right).\end{aligned}\tag{6.8}$$

Assim, a desigualdade de Chebyshev dá

$$\mathbb{P}[\,|X_i - c| \geqslant t\,] \leqslant \frac{c(1-1/n)}{t^2} \leqslant \frac{c}{t^2},$$

e, por exemplo, tomando $t = c$,

$$\mathbb{P}[\,X_i \geqslant 2c\,] \leqslant \mathbb{P}[\,|X_i - c| \geqslant c\,] \leqslant \frac{1}{c} - \frac{1}{cn}.$$

O que só é útil para c grande.

Pela cota da união sobre as n listas,

$$\mathbb{P}\left[\,\max_i X_i \geqslant c + t\,\right] \leqslant n\cdot\frac{c}{t^2} = \frac{m}{t^2},$$

e, com $t = \sqrt{2m}$, a maior lista tem tamanho no máximo $c + \sqrt{2m} = O(\sqrt{m})$ com probabilidade pelo menos $1/2$ para $n = \Omega(\sqrt{m})$.

O número esperado de colisões é $\mathbb{E}C = \binom{m}{2}/n$ (eq. (6.6)), onde

$$C = \sum_{\substack{\{x,y\}\subseteq S\\ x\neq y}}\mathbb{1}_{[h(x)=h(y)]}$$

conta os pares não ordenados que colidem.

Para pares distintos $\{x,y\} \neq \{z,w\}$ em S, os indicadores de colisão são independentes. Se os pares são disjuntos, eles dependem de quatro valores independentes. Se compartilham um elemento, digamos $\{x,y\}$ e $\{x,z\}$, então

$$\mathbb{P}[\, h(x) = h(y) = h(z)\,] = \sum_{a \in N} \frac{1}{n^3} = \frac{1}{n^2} = \mathbb{P}[\, h(x) = h(y)\,]\mathbb{P}[\, h(x) = h(z)\,]. \tag{6.9}$$

Logo os indicadores são independentes dois a dois e a variância da soma é a soma das variâncias. Cada um tem variância $\frac{1}{n} - \frac{1}{n^2}$ e, portanto,

$$\operatorname{Var}(\, C\,) = \sum_{\substack{\{x,y\} \subseteq S \\ x \neq y}} \operatorname{Var}\big(\, \mathbb{1}_{[h(x)=h(y)]}\,\big) = \binom{m}{2}\left(\frac{1}{n} - \frac{1}{n^2}\right).$$

Como consequência, a desigualdade de Chebyshev dá

$$\mathbb{P}[\, C = 0\,] \leqslant \mathbb{P}[\, |C - \mathbb{E}C| \geqslant \mathbb{E}C\,] \leqslant \frac{\operatorname{Var}(\, C\,)}{(\mathbb{E}C)^2} = \frac{n(1-1/n)}{\binom{m}{2}} \approx \frac{2n}{m^2},$$

de modo que, para $m \gg \sqrt{n}$, há colisão com alta probabilidade. Isso complementa o limiar $m \approx \sqrt{n}$ do paradoxo dos aniversários.

Ideia importante

Vimos que para as estimativas de interesse para colisões basta controlar a colisão entre pares: qualquer família em que dois elementos distintos colidem com probabilidade no máximo $1/n$ tem, no máximo, o mesmo número esperado de colisões que uma função aleatória.
Para uma função aleatória, a probabilidade de colisão de um par é exatamente $1/n$, e foi só isso que usamos para o número esperado de colisões. Basta, portanto, que para uma família $\mathcal{H} \subseteq N^U$ de funções valha $\mathbb{P}_{h \in_R \mathcal{H}}[\, h(x)=h(y)\,] \leqslant \frac{1}{n}$ para todos $x \neq y$ em U. Já para a variância do tamanho da lista, a independência par a par dos indicadores $\mathbb{1}_{[h(x)=i]}$ também é suficiente. Isso é útil porque funções aleatórias não são práticas: não têm descrição pequena, e armazená-las exigiria espaço proporcional a $|U|$. É possível, como veremos, sortear h de uma família menor cujas funções são fáceis de computar, têm descrição curta e preservam as propriedades estatísticas de que precisamos. Note que se comportar como uma função aleatória é uma propriedade da família, não de uma função isolada.

Observação 16. A variância de X_i e, consequentemente, a cota $c + \sqrt{2m}$ para a maior lista usam apenas independência dois a dois e, portanto, valem também para as famílias da Seção 7; em contraste com

o $\Theta(\log n / \log\log n)$, que usa independência de muitas variáveis ao mesmo tempo, esse é o preço da representação sucinta.

Já a fórmula para $\mathrm{Var}(\,C\,)$ usa, por meio de (6.9), a independência de três e de quatro valores $h(x)$ ao mesmo tempo. Ela vale para a função totalmente aleatória (e para famílias com independência de ordem 4), mas não para famílias apenas 2-universais, para as quais só se garante $\mathbb{E}C = \binom{m}{2}/n$.

6.2 *Hashing universal*

Idealmente, o que procuramos para funções de espalhamento são funções que tenham representação sucinta e imitem uma função aleatória nas estatísticas de interesse, como tempo constante de busca e a ausência de listas longas, e que também possam ser computadas eficientemente em cada ponto do domínio.

A ideia principal para resolver o problema do armazenamento de funções é reduzir a aleatoriedade na escolha de h. Formalmente, em vez de selecionar h de maneira aleatória em N^U, sorteamos uma função de uma coleção menor e não vazia de funções, $\mathcal{H} \subseteq N^U$. Em vez de exigirmos que a equação (6.1) seja válida para todo x e todo i, pedimos que seja válida a condição

$$\mathop{\mathbb{P}}_{h \in_R \mathcal{H}} [\, h(x) = h(y) \,] \leqslant \frac{1}{n}, \tag{6.10}$$

para todos $x \neq y$ em U. Uma família não vazia $\mathcal{H} \subseteq N^U$ de funções que satisfazem a equação (6.10) é chamada de ***universal***.

Para uma família universal, ainda valem, como desigualdades, as principais conclusões obtidas para funções aleatórias:

- o tamanho esperado da lista $H[h(x)]$ é no máximo c se $x \notin S$ e no máximo $1 + \frac{m-1}{n} \leqslant 1 + c$ se $x \in S$, pois a conta em (6.4) usa apenas $\mathbb{P}[\, h(x) = h(y) \,] \leqslant 1/n$;
- os tempos esperados de busca da Proposição 6.1 são *no máximo* $c+1$ (sem sucesso) e $1 + \frac{c}{2} - \frac{1}{2n}$ (com sucesso), pois a prova usa apenas a probabilidade de colisão de pares e o fato de que a ordem de inserção não depende de h;
- o número esperado de colisões é no máximo $\binom{m}{2}\frac{1}{n} = \frac{m(m-1)}{2n}$ e, no caso $n = m^2$, esse valor é menor que $1/2$, de modo que, por (3.5), não há colisão com probabilidade maior que $1/2$ (verifique).

Teorema 6.1: colisões para *hashing* universal

Se $\mathcal{H}$ é universal, então para qualquer conjunto $S \subseteq U$ de cardinalidade m e para qualquer $x \in U$, se h é escolhido aleatoria-

mente em $\mathcal{H}$, então o número esperado de colisões entre x e os elementos de $S \setminus \{ x \}$ é no máximo m/n.

Demonstração. Pela linearidade da esperança e por (6.10),

$$\mathbb{E} \sum_{y \in S \setminus \{ x \}} \mathbb{1}_{[h(x)=h(y)]} = \sum_{y \in S \setminus \{ x \}} \mathbb{P}[\, h(x) = h(y) \,] \leqslant \frac{|S \setminus \{ x \}|}{n} \leqslant \frac{m}{n}. \qquad \square$$

Como não precisamos de funções genuinamente aleatórias para termos as boas propriedades estatísticas dessas funções, vamos procurar por uma família universal de funções de *hash* com representações sucintas e que sejam calculadas eficientemente. Por exemplo, a família $\{ f_1, f_2, f_3 \}$ de funções de $\{ 1, 2 \}$ em $\{ 0, 1 \}$ dadas na Tabela 6.1 é universal (verifique). Note que nessa família a equação (6.1) não vale:

	1	2
f_1	0	0
f_2	1	0
f_3	0	1

Tabela 6.1: exemplo de uma família universal.

$\mathbb{P}[\, f(1) = 0 \,] = 2/3 \neq 1/2$. Ou seja, universal não implica que cada $h(x)$ seja uniforme em N.

Exemplo 6.1: uma família universal linear

Podemos definir uma função linear

$$\begin{aligned} h \colon \{ 0,1 \}^d &\to \{ 0,1 \}^k \\ \mathsf{x} &\mapsto \mathsf{A} \cdot \mathsf{x} \end{aligned}$$

sorteando os bits de uma matriz $\mathsf{A} = (a_{\ell,j})$ de dimensão $k \times d$ e tomando as operações de soma e multiplicação módulo 2. Fazendo $U := \{ 0,1 \}^d$ e $N := \{ 0,1 \}^k$, com $|U| = 2^d$ e $|N| = 2^k$, para armazenar a função precisamos apenas da matriz A, que tem $kd = O(\log |N| \cdot \log |U|)$ bits.
Nesse caso, a equação (6.1) não vale para todo x e todo i, pois $h(\mathsf{0}) = \mathsf{0}$ para qualquer escolha de A.

A família é universal: sejam $\mathsf{x}, \mathsf{y} \in U$ distintos. Como h é linear, $h(\mathsf{x}) = h(\mathsf{y})$ se, e somente se, $\mathsf{A} \cdot (\mathsf{x} - \mathsf{y}) = \mathsf{0}$, isto é,

$$\sum_{j=1}^{d} a_{\ell,j} (\mathsf{x} - \mathsf{y})_j = 0 \text{ para cada } \ell \in \{ 1, \ldots, k \}.$$

Seja r uma coordenada não nula de $\mathsf{x} - \mathsf{y}$, ou seja, $(\mathsf{x} - \mathsf{y})_r = 1$ (em aritmética módulo 2). Fixe todos os bits de A fora da coluna r. Então, para cada ℓ, a equação acima se verifica se, e

somente se,

$$a_{\ell,r} = \sum_{\substack{j=1 \\ j \neq r}}^{d} a_{\ell,j}(\mathsf{x}-\mathsf{y})_j \pmod 2,$$

que determina um único valor para o bit $a_{\ell,r}$. Como os k bits da coluna r são sorteados uniformemente e de forma independente, essa condição vale para todas as k linhas com probabilidade 2^{-k}, qualquer que seja a escolha dos demais bits. Portanto,

$$\mathop{\mathbb{P}}_{\mathsf{A} \in_{\mathrm{R}} \{0,1\}^{k \times d}} [\, \mathsf{A}\cdot\mathsf{x} = \mathsf{A}\cdot\mathsf{y} \,] = \frac{1}{2^k} = \frac{1}{|N|}.$$

Carter e Wegman (1979) introduziu o conceito de *hashing universal*: em vez de exigir independência total entre as imagens de todas as chaves, o que consome muitos bits aleatórios, basta que a probabilidade de colisão entre quaisquer duas chaves distintas seja no máximo $1/n$. Famílias com a propriedade mais forte de independência de pares (Seção 7) também são universais, mas a recíproca é falsa.

Exemplo 6.2: o exemplo canônico de uma família universal

Tomamos $U := \{0, 1, \ldots, u-1\}$ e $N := \{0, 1, \ldots, n-1\}$, com $u > n$, e fixamos um primo p com $u < p \leqslant 2u$ (que existe pelo postulado de Bertrand). Definimos a família de funções

$$\mathcal{H}_{p,n} := \left\{ h_{(a,b)} : a, b \in \{0, 1, \ldots, p-1\},\ a \neq 0 \right\}$$

com $h_{(a,b)}(x) = \big((ax + b) \bmod p\big) \bmod n$. Sortear uma função de $\mathcal{H}_{p,n}$ significa sortear (a, b) uniformemente em $\{1, \ldots, p-1\} \times \{0, \ldots, p-1\}$. A restrição $a \neq 0$ é essencial: com $a = 0$ a função seria constante e todos os pares colidiriam. Sejam $x \neq y$ em U. Como $u < p$, temos $0 < |x - y| < p$ e, sendo p primo, $p \nmid x - y$. Se $a \neq 0$, então $p \nmid a(x - y)$, ou seja,

$$(ax + b) \bmod p \neq (ay + b) \bmod p.$$

Ademais, para quaisquer $i, j \in \{0, 1, \ldots, p-1\}$ distintos existe exatamente um par (a, b) com $a \neq 0$ tal que

$$\begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \equiv \begin{pmatrix} i \\ j \end{pmatrix} \pmod p, \tag{6.11}$$

pois a matriz é invertível módulo p (seu determinante $x - y$ não é divisível por p). A solução tem $a \neq 0$ justamente porque $i \neq j$: subtraindo as equações, $a(x - y) \equiv i - j \not\equiv 0$. São $p(p-1)$ pares (i, j) distintos e o mesmo tanto de pares (a, b) com $a \neq 0$,

logo há uma bijeção entre esses pares. Disso tiramos que, para $x \neq y$ dados, se sorteamos (a, b) uniformemente, o par $(i, j) = \big((ax + b) \bmod p, (ay + b) \bmod p\big)$ resultante é qualquer par de valores distintos módulo p com a mesma probabilidade. Resta contarmos a quantidade de pares (i, j) de números distintos e congruentes módulo n.
Para cada $i \in \{0, 1, \ldots, p-1\}$, os $j \neq i$ em $\{0, 1, \ldots, p-1\}$ com $j \equiv i \pmod{n}$ correspondem aos múltiplos *não nulos* de n entre os p inteiros consecutivos $\{-i, \ldots, -i + p - 1\}$ (o múltiplo 0 corresponde a $j = i$). Entre p inteiros consecutivos há no máximo $\lceil p/n \rceil \leqslant (p-1)/n + 1$ múltiplos de n, logo há no máximo $(p-1)/n$ valores de j. Assim, há no máximo $p(p-1)/n$ pares (i, j) distintos e congruentes módulo n, e para $x \neq y$

$$\mathbb{P}\big[\, h_{(a,b)}(x) = h_{(a,b)}(y) \,\big] \leqslant \frac{p(p-1)/n}{p(p-1)} = \frac{1}{n}.$$

Como p é fixo e conhecido, cada função é descrita apenas pelo par (a, b), com $2\lceil \log p \rceil \leqslant 2 \log u + O(1) = O(\log |U|)$ bits. No modelo RAM, com palavras de $\Theta(\log u)$ bits, a função consiste em um número constante de operações aritméticas (uma multiplicação, uma soma e dois restos), logo pode ser calculada em tempo constante.

6.2.1 *Hashing perfeito*

Dizemos que uma função de *hash* é ***perfeita*** para um conjunto *estático* S se não há colisões em S, isto é, h é injetora em S. Nesse caso, cada lista tem no máximo um elemento e uma busca custa $O(1)$ no pior caso.

Supomos $|U| > m^2$, para que as famílias do Exemplo 6.2 possam ser usadas com contradomínios de até m^2 posições.

Primeira construção Se $\mathcal{H}$ é universal e $n = m^2$, então o número C de colisões em S satisfaz

$$\mathbb{E}C \leqslant \binom{m}{2} \frac{1}{m^2} = \frac{m(m-1)}{2m^2} < \frac{1}{2},$$

e, por (3.5), $\mathbb{P}[\, C \geqslant 1 \,] \leqslant \mathbb{E}C < \frac{1}{2}$. Logo, escolhendo h aleatoriamente em $\mathcal{H}$, a probabilidade de não haver colisões é maior que $1/2$ (veja (6.6) e (6.7)). Isso sugere um algoritmo do tipo Las Vegas, supondo que $h(x)$ é calculado em tempo $O(1)$:

1. sorteie h em $\mathcal{H}$;
2. teste todos os $O(m^2)$ pares de elementos de S para colisão;

3. repita os passos 1 e 2 até encontrar h sem colisões.

Cada tentativa custa $O(m^2) = O(n)$ e tem probabilidade de sucesso maior que $1/2$. O número de tentativas é uma variável aleatória geométrica de média menor que 2, portanto a função perfeita é encontrada em tempo *esperado* $O(n)$. Dessa forma, temos uma tabela de *hashing* para um conjunto fixo de tamanho m, com tempo de busca $O(1)$ no pior caso e espaço $O(m^2)$.

Segunda construção (espaço linear) É possível obter tempo de busca constante no pior caso usando apenas espaço $O(m)$. O método tem dois níveis:

- *primeiro nível:* espalhamos S em uma tabela H de tamanho m usando uma função h de uma família universal (digamos $h(x) = ((ax + b) \bmod p) \bmod m$), o que poderá resultar em colisões;
- *segundo nível:* se m_i elementos vão parar na posição i de H, eles são distribuídos em uma tabela secundária H_i com m_i^2 posições, usando uma função h_i sem colisões (digamos $h_i(x) = ((a_i x + b_i) \bmod p) \bmod m_i^2$), como na primeira construção. Se $m_i = 0$ não há tabela e, se $m_i = 1$, a tabela tem uma posição.

Para buscar x, calculamos $i = h(x)$ e procuramos o elemento em $H_i[h_i(x)]$, o que custa $O(1)$ no pior caso.

O espaço usado no segundo nível é $\sum_i m_i^2$. Como $m_i^2 = m_i + m_i(m_i - 1) = m_i + 2\binom{m_i}{2}$ e $\sum_i m_i = m$,

$$\mathbb{E} \sum_{i=0}^{m-1} m_i^2 = m + 2\,\mathbb{E} \sum_{i=0}^{m-1} \binom{m_i}{2}.$$

Aqui $\binom{m_i}{2}$ conta os pares não ordenados de elementos que colidem na posição i da tabela de primeiro nível, portanto $\sum_i \binom{m_i}{2}$ é o número de colisões no *hashing* primário, e cada par que colide é contado exatamente uma vez. Como a probabilidade de um dado par colidir é no máximo $1/m$,

$$\mathbb{E} \sum_i \binom{m_i}{2} \leqslant \frac{1}{m}\binom{m}{2} = \frac{m-1}{2},$$

e portanto $\mathbb{E} \sum_i m_i^2 \leqslant m + (m-1) < 2m$. Pela desigualdade de Markov,

$$\mathbb{P}\left[\sum_i m_i^2 \geqslant 4m \right] \leqslant \frac{\mathbb{E} \sum_i m_i^2}{4m} < \frac{2m}{4m} = \frac{1}{2},$$

ou seja, com probabilidade maior que $1/2$ a soma dos tamanhos das tabelas secundárias é menor que $4m$.

Construção. Para encontrar h com $\sum_i m_i^2 < 4m$, usamos o mesmo procedimento de tentativa e repetição da primeira construção. Para testar uma escolha de h, basta calcular os valores m_i, o que é feito em tempo $O(m)$ percorrendo os m elementos de S. A probabilidade de sucesso em cada tentativa é maior que $1/2$, logo o número de tentativas é geométrico de média menor que 2 e o tempo esperado para encontrar h é $O(m)$.

Fixada essa função de primeiro nível, para cada i com $m_i \geqslant 1$ sorteamos h_i até que não haja colisões entre os m_i elementos que caem na posição i. Pela primeira construção, o tempo esperado para construir h_i é $O(m_i^2)$. Como $\sum_i m_i^2 < 4m$ vale *por construção*, o tempo esperado total para construir as tabelas secundárias é $O\left(\sum_i m_i^2\right) = O(m)$; a aleatoriedade restante está apenas nos sorteios dos h_i.

Por fim, o espaço total é m (tabela primária) mais $\sum_i m_i^2 < 4m$ (tabelas secundárias) mais $O(1)$ palavras para os parâmetros (a, b) e (a_i, b_i) de cada tabela, ou seja, $O(m)$. Resumindo: para um conjunto estático S de tamanho m, construímos em tempo esperado $O(m)$ uma estrutura de espaço $O(m)$ com busca em tempo $O(1)$ no pior caso.

6.2.2 *Aplicação em streaming: Count-Min sketch*

Suponha que itens $x_1, x_2, \ldots, x_m \in U$ chegam um a um, como num fluxo de dados (acessos a páginas, pacotes numa rede), e que queremos, a qualquer momento, estimar a *frequência* $f(x) \coloneqq |\{\, t : x_t = x \,\}|$ de um item $x \in U$. Guardar um contador para cada item de U exige espaço $|U|$, que é proibitivo. Armazenar os m itens também pode ser proibitivo. O ***Count-Min sketch*** usa funções de *hash* universais para resumir o fluxo em uma pequena matriz de contadores. Note que $\sum_{y \in U} f(y) = m$ e que o fluxo é fixo, isto é, não depende das funções sorteadas.

Sejam $n, d \geqslant 1$ inteiros e $h_1, \ldots, h_d : U \to \{0, 1, \ldots, n-1\}$ funções escolhidas *independentemente* e uniformemente em uma família universal $\mathcal{H}$ (por exemplo, a do Exemplo 6.2). Mantemos uma matriz de contadores (*sketch*) $C[j][k]$, com $1 \leqslant j \leqslant d$ e $0 \leqslant k < n$, inicialmente nulos.

- *chegada de x:* para cada j, incrementamos $C[j][h_j(x)]$;
- *consulta de x:* devolvemos $\widehat{f}(x) \coloneqq \min_{1 \leqslant j \leqslant d} C[j][h_j(x)]$.

Proposição 6.2: Count-Min sketch

Para todo $x \in U$ vale $\widehat{f}(x) \geqslant f(x)$. Além disso, dados $\varepsilon, \delta \in (0,1)$, se $n = \lceil 2/\varepsilon \rceil$ e $d = \lceil \log_2(1/\delta) \rceil$, então

$$\mathbb{P}\left[\, \widehat{f}(x) > f(x) + \varepsilon m \,\right] \leqslant \delta.$$

O *sketch* usa $nd = O\left(\frac{1}{\varepsilon}\log\frac{1}{\delta}\right)$ contadores, cada um com $O(\log m)$ bits. Para a família do Exemplo 6.2, cada uma das d funções se descreve com $O(\log|U|)$ bits.

Demonstração. Fixemos $x \in U$ e, para cada linha j, seja $C_j \coloneqq C[j][h_j(x)]$. Cada chegada de x incrementa C_j, de modo que $C_j \geqslant f(x)$ para todo j, e então $\widehat{f}(x) \geqslant f(x)$. O ***erro*** da linha j é

$$E_j \coloneqq C_j - f(x) = \sum_{y \neq x} f(y)\,\mathbb{1}_{[h_j(y)=h_j(x)]} \geqslant 0,$$

pois é a soma das frequências dos outros itens que colidem com x na linha j. Como $\mathcal{H}$ é universal, pela linearidade da esperança

$$\mathbb{E}E_j = \sum_{y \neq x} f(y)\,\mathbb{P}\left[\, h_j(y) = h_j(x) \,\right] \leqslant \frac{1}{n}\sum_{y \neq x} f(y) \leqslant \frac{m}{n},$$

em que usamos $\sum_y f(y) = m$. Como $E_j \geqslant 0$, a desigualdade de Markov (3.5) dá

$$\mathbb{P}\left[\, E_j > \varepsilon m \,\right] \leqslant \mathbb{P}\left[\, E_j \geqslant \varepsilon m \,\right] \leqslant \frac{\mathbb{E}E_j}{\varepsilon m} \leqslant \frac{1}{\varepsilon n} \leqslant \frac{1}{2},$$

pois $n \geqslant 2/\varepsilon$. Agora, $\widehat{f}(x) = f(x) + \min_j E_j$, de modo que $\widehat{f}(x) > f(x) + \varepsilon m$ se, e somente se, $E_j > \varepsilon m$ para *todo* j. Como o fluxo é fixo e E_j depende apenas de h_j, as funções independentes $h_1, \ldots, h_d$ tornam os eventos $[E_j > \varepsilon m]$ independentes, e

$$\mathbb{P}\left[\, \widehat{f}(x) > f(x) + \varepsilon m \,\right] = \prod_{j=1}^{d} \mathbb{P}\left[\, E_j > \varepsilon m \,\right] \leqslant 2^{-d} \leqslant \delta.$$

A contagem de espaço segue dos valores de n e d, e de que cada contador nunca excede m. □

A estimativa nunca é menor que a frequência verdadeira (o erro é unilateral) e, com probabilidade pelo menos $1 - \delta$, excede-a em no máximo εm. O espaço não depende de $|U|$ nem de m (exceto pelos $\log m$ bits de cada contador), e a escolha de d ilustra o padrão que vimos no hashing perfeito: uma probabilidade de falha constante em cada tentativa, reduzida exponencialmente por repetições independentes.

Observação 17. Dentro de cada h_j, a demonstração usa apenas a universalidade, $\mathbb{P}[\, h(x) = h(y) \,] \leqslant 1/n$, e a desigualdade de Markov; não é necessária a independência de pares. Já a independência ***entre*** $h_1, \ldots, h_d$ é essencial para a cota 2^{-d}. O erro é proporcional ao comprimento m do fluxo, e não a $f(x)$, de modo que a estimativa *só é informativa para itens frequentes*, com $f(x) \gg \varepsilon m$, que é o caso de interesse (por exemplo, na detecção dos itens mais acessados). A análise acima supõe que os itens apenas chegam; se houver remoções, a propriedade $\widehat{f}(x) \geqslant f(x)$ pode falhar e a análise é outra. ◊

6.2.3 *Aplicação: filtro de Bloom*

Suponha que queremos representar um conjunto $S \subseteq U$ com $m = |S|$ elementos e responder a consultas "$x \in S$?". Uma tabela de espalhamento resolve o problema, mas guarda os próprios elementos e, portanto, usa pelo menos $\log_2 \binom{|U|}{m} \geqslant m \log_2(|U|/m)$ bits. Em muitas aplicações, como decidir se um endereço está numa lista de páginas maliciosas ou se uma chave está num arquivo em disco antes de pagar o custo do acesso, é aceitável errar *num único sentido*: ocasionalmente responder "sim" para um $x \notin S$ (um ***falso positivo***), mas nunca responder "não" para um $x \in S$ (um ***falso negativo***). Em troca do erro unilateral, ganhamos espaço: poucos bits por elemento, sem depender do tamanho dos elementos. É o que faz o ***filtro de Bloom*** (Bloom, 1970).

Sejam $n, k \geqslant 1$ inteiros e $h_1, \ldots, h_k : U \to \{0, 1, \ldots, n-1\}$ funções escolhidas *independentemente* e uniformemente em uma família universal $\mathcal{H}$ (por exemplo, a do Exemplo 6.2). Mantemos uma matriz de bits $B[j][l]$, com $1 \leqslant j \leqslant k$ e $0 \leqslant l < n$, inicialmente nulos. A estrutura é a do *Count-Min sketch* com contadores de um bit:

- *inserção de x:* para cada j, fazemos $B[j][h_j(x)] \leftarrow 1$;
- *consulta de x:* devolvemos 1 ("talvez $x \in S$") se $B[j][h_j(x)] = 1$ para *todo* j e 0 ("certamente $x \notin S$") caso contrário.

O espaço é kn bits, mais $O(\log |U|)$ bits para descrever cada uma das k funções (para a família do Exemplo 6.2).

Proposição 6.3: filtro de Bloom

Sejam $S \subseteq U$ com $|S| = m$ e $x \in U$ fixados *antes* do sorteio das funções, e suponha que todos os elementos de S foram inseridos num filtro de Bloom com k linhas de n bits, em que $h_1, \ldots, h_k$ são escolhidas independentemente e uniformemente em uma família universal com contradomínio $\{0, \ldots, n-1\}$ (a do Exemplo 6.2 exige $|U| > n$). Então

1. se $x \in S$, a consulta devolve 1 com probabilidade 1, para qualquer escolha das funções;
2. se $x \notin S$, a consulta devolve 1 com probabilidade no máximo $(m/n)^k$.

Em particular, dado $\delta \in (0, 1)$, se $n = 2m$ e $k = \lceil \log_2(1/\delta) \rceil$, a probabilidade de falso positivo é no máximo δ e o espaço é $kn = 2m\lceil \log_2(1/\delta) \rceil$ bits, além dos $O(\log |U| \log(1/\delta))$ bits das funções (na família do Exemplo 6.2).

Demonstração. Se $x \in S$, a inserção de x colocou 1 em todas as posições $B[j][h_j(x)]$, e nenhum bit volta a ser 0; logo a consulta devolve 1.

Fixemos $x \notin S$ e uma linha j. Como só a função h_j escreve na linha j, temos $B[j][h_j(x)] = 1$ se, e somente se, existe $y \in S$ com $h_j(y) = h_j(x)$; chamemos esse evento de E_j. Como $x \neq y$ para todo $y \in S$, a cota da união e a universalidade de $\mathcal{H}$ dão

$$\mathbb{P}\big[\, E_j \,\big] \leqslant \sum_{y \in S} \mathbb{P}\big[\, h_j(y) = h_j(x) \,\big] \leqslant \frac{m}{n}.$$

Os conjuntos S e $\{\, x \,\}$ estão fixos e E_j depende apenas de h_j; como as funções são independentes, os eventos $E_1, \ldots, E_k$ são independentes, e a consulta devolve 1 se, e somente se, todos ocorrem. Portanto

$$\mathbb{P}\big[\, \text{falso positivo} \,\big] = \prod_{j=1}^{k} \mathbb{P}\big[\, E_j \,\big] \leqslant \left(\frac{m}{n}\right)^{k}.$$

Para $n = 2m$ essa cota é $2^{-k} \leqslant \delta$. □

O erro é *unilateral*, como no protocolo de impressões digitais (Seção 7.1), e a análise segue o padrão que já vimos no *Count-Min sketch* e no *hashing* perfeito: uma probabilidade de falha de no máximo $1/2$ em cada linha (com $n = 2m$), obtida com a cota da união e a universalidade, é reduzida exponencialmente por $k = \log_2(1/\delta)$ repetições independentes. De fato, o filtro é um *Count-Min sketch* com contadores de um bit que saturam em 1: a consulta, que no *Count-Min* é o mínimo dos contadores, passa a ser a conjunção (AND) dos bits. A cota só é informativa se $n > m$, isto é, com mais de um bit por elemento em cada linha.

A Proposição trata de x fixo, e a probabilidade é sobre a escolha das funções. Na prática, porém, o filtro é construído uma vez e depois consultado muitas vezes. O corolário seguinte diz que, para a maioria das escolhas das funções, a *fração* de consultas que erram também é pequena.

Corolário 6.1: taxa de falsos positivos

Sejam S e o filtro como na Proposição 6.3 e seja Q uma distribuição de probabilidade sobre $U \setminus S$, fixada independentemente de $h_1, \ldots, h_k$ (por exemplo, a uniforme). Seja Φ a probabilidade, segundo Q, de que a consulta de $x \sim Q$ devolva 1, isto é, a taxa de falsos positivos do filtro construído. Então $\mathbb{E}\Phi \leqslant (m/n)^k$ e, para todo $c > 1$,

$$\mathbb{P}\big[\, \Phi \geqslant c\,(m/n)^k \,\big] \leqslant \frac{1}{c}.$$

Demonstração. Se $\mathbb{1}_{[x \text{ é falso positivo}]}$ vale 1 quando a consulta de x devolve 1, então $\Phi = \sum_{x \notin S} Q(x)\, \mathbb{1}_{[x \text{ é falso positivo}]}$ e, pela linearidade da esperança e pela Proposição 6.3 aplicada a cada $x \notin S$, $\mathbb{E}\Phi = \sum_{x \notin S} Q(x) \mathbb{P}\left[x \text{ é falso positivo} \right] \leqslant (m/n)^k$. Como $\Phi \geqslant 0$, a desigualdade de Markov dá o resto. □

Por exemplo, com $n = 2m$ e $k = \lceil \log_2(1/\delta) \rceil$, com probabilidade pelo menos 0,9 sobre a escolha das funções a taxa de falsos positivos do filtro construído é no máximo $10\,\delta$.

A variante clássica: um único vetor No filtro de Bloom original as k funções escrevem num mesmo vetor B de n bits, com $h_j \colon U \to \{0, 1, \ldots, n-1\}$, em vez de em k linhas separadas de n bits cada; aqui n passa a designar o espaço total do filtro. A análise exata pede mais do que universalidade, pois os bits de um mesmo vetor deixam de ser independentes entre as sondagens. Aqui fazemos a conta no modelo de funções totalmente aleatórias, como na Seção 6.1.

Depois de inserir m elementos foram feitas km escolhas independentes e uniformes de posição em n bits. Seja X o número de bits iguais a 1 no vetor. Cada bit continua nulo com probabilidade $(1 - 1/n)^{km}$, de modo que

$$\mathbb{E}X = n\left(1 - \left(1 - \frac{1}{n}\right)^{km}\right).$$

Para $x \notin S$, as k posições sondadas são independentes e uniformes, e independentes do vetor. Dado o vetor, a consulta devolve 1 se, e somente se, todas as k posições sondadas são bits 1, o que ocorre com probabilidade $(X/n)^k$. Como X é aleatório e os bits do vetor não são independentes, a probabilidade de falso positivo é a esperança $\mathbb{E}(X/n)^k$, e não o produto das probabilidades marginais:

$$\mathbb{P}\left[\text{falso positivo} \right] = \mathbb{E}\left(\frac{X}{n}\right)^k \geqslant \left(\frac{\mathbb{E}X}{n}\right)^k = \left(1 - \left(1 - \frac{1}{n}\right)^{km}\right)^k > \left(1 - e^{-km/n}\right)^k,$$

em que a primeira desigualdade é a de Jensen ($t \mapsto t^k$ é convexa) e a última vem de $1 - 1/n < e^{-1/n}$. Portanto a fórmula clássica

$$\left(1 - e^{-km/n}\right)^k \tag{6.12}$$

é um limitante *inferior* para a probabilidade de falso positivo e, para k fixo e m/n fixo, com n grande, é conhecido da literatura que a diferença para o valor exato é desprezível (resultado que não provamos). Assim, a fórmula serve para dimensionar o filtro, com a ressalva de que o erro real é um pouco maior.

Qual k minimiza (6.12), fixados n e m? Escrevendo $u = e^{-km/n}$, de modo que $k = -\frac{n}{m}\ln u$, temos

$$\left(1 - e^{-km/n}\right)^k = \exp\left(-\frac{n}{m} f(u)\right), \qquad f(u) := \ln u \cdot \ln(1-u) > 0.$$

Minimizar (6.12) equivale a maximizar f em $(0,1)$. A função f é simétrica ($f(u) = f(1-u)$) e estritamente côncava[2], portanto tem um único máximo, em $u = 1/2$, onde $f = (\ln 2)^2$. Isso ocorre para

$$k = \frac{n}{m}\ln 2,$$

(que na prática é arredondado para um inteiro). Nesse valor, metade dos bits do vetor é 1 e

$$\left(1 - \tfrac{1}{2}\right)^k = 2^{-k} = 2^{-(n/m)\ln 2}.$$

Para obter probabilidade de falso positivo (aproximadamente) δ tomamos $k = \log_2(1/\delta)$ e

$$n = \frac{m \log_2(1/\delta)}{\ln 2} = (\log_2 e)\, m \log_2(1/\delta) \approx 1{,}44\, m \log_2(1/\delta)$$

bits, contra os $2m\lceil \log_2(1/\delta)\rceil$ da Proposição 6.3. Como o erro real é um pouco maior que (6.12), na prática escolhe-se n ligeiramente maior. Em outras palavras, o espaço por elemento depende só do erro desejado δ, e não de m nem de $|U|$: cerca de 9,6 bits por elemento para $\delta = 1\%$ (com $k = 7$) e cerca de 14,4 bits para $\delta = 0{,}1\%$ (com $k = 10$).

[2] Seja $F(u) := 2u(1-u) + (1-u)^2 \ln(1-u) + u^2 \ln u$. Um cálculo direto dá $f''(u) = -F(u)/\big(u^2(1-u)^2\big)$. Temos $F(0) = F(1) = 0$ (pelos limites) e $F''(u) = 2\ln(u(1-u)) + 2 \leqslant 2\ln\frac{1}{4} + 2 < 0$, pois $u(1-u) \leqslant \frac{1}{4}$. Logo F é estritamente côncava e, como se anula nos extremos, $F > 0$ em $(0,1)$, isto é, $f'' < 0$.

Exemplo 6.3: comparação de espaço

Seja $m = 10^8$ endereços de 50 bytes (400 bits cada) e $\delta = 10^{-2}$. A tabela compara o espaço necessário.

estrutura	bits	aprox.
guardar os endereços	4×10^{10}	5 GB
guardar impressões de 34 bits	$3{,}4 \times 10^9$	425 MB
Bloom em k linhas, $n = 2m$, $k = 7$	$1{,}4 \times 10^9$	175 MB
Bloom em vetor único, $k = 7$	$9{,}6 \times 10^8$	120 MB

Na segunda linha, cada endereço é substituído por uma impressão digital de $r = \lceil \log_2(m/\delta)\rceil = 34$ bits, calculada por uma função de uma família *universal* (por exemplo, a do Exemplo 6.2, com contradomínio de 2^r valores), cuja descrição, de $O(\log |U|)$ bits, é paga uma única vez e é desprezível diante de mr. Pela cota da união, a probabilidade de um $x \notin S$ coincidir com alguma impressão é no máximo $m2^{-r} \leqslant \delta$. O custo mr cresce com $\log m$, enquanto o do filtro de Bloom é de poucos bits por elemento; além disso, mr não inclui a estrutura

necessária para guardar e consultar as impressões. Com as famílias ε-quase universais da Seção 7.1, que têm descrição ainda mais curta mas colisão por par de $\approx m_0/p$, onde $m_0{=}400$ é o número de bits de cada endereço, seriam necessários cerca de $\log_2(m\, m_0/\delta){\approx}42$ bits por impressão.

Observação 18. Qualquer estrutura que não tenha falsos negativos e tenha probabilidade de falso positivo no máximo δ precisa, para $m \ll |U|$, de cerca de $m \log_2(1/\delta)$ bits (resultado clássico, que não provamos). O filtro de Bloom em vetor único está, portanto, a um fator $\log_2 e \approx 1{,}44$ do ótimo.

Algumas propriedades práticas: (i) a união de dois filtros com as mesmas funções e o mesmo tamanho é o OR bit a bit, e é o filtro da união dos conjuntos; (ii) *não* há como remover um elemento, pois zerar um bit pode apagar outros elementos e criar falsos negativos. A variante com remoção troca cada bit por um contador, incrementado na inserção e decrementado na remoção, o que a aproxima ainda mais do *Count-Min sketch*; (iii) a garantia vale para até m elementos, e inserir muito mais que isso degrada a probabilidade de falso positivo; (iv) a estrutura não permite listar os elementos de S. ◊

Aplicações O filtro de Bloom é usado sempre que uma consulta negativa barata evita uma operação cara. Em bancos de dados baseados em arquivos ordenados em disco, como Bigtable, HBase e Cassandra, cada arquivo mantém um filtro de suas chaves em memória; antes de ler o arquivo, consulta-se o filtro, e a resposta "certamente não" dispensa o acesso a disco. Os falsos positivos custam apenas uma leitura desnecessária, enquanto a ausência de falsos negativos garante a correção do resultado. O mesmo princípio vale para *caches* de rede e de conteúdo (verificar se um objeto já foi visto ou armazenado antes de buscá-lo), para *crawlers* (descartar endereços já visitados sem guardar a lista completa) e para junções em bancos de dados distribuídos, em que um nó envia a outro um filtro com as chaves de uma tabela, para que apenas as linhas candidatas sejam transmitidas, reduzindo o tráfego. Na bioinformática, filtros armazenam conjuntos enormes de subcadeias de tamanho fixo (*k-mers*) de genomas, em espaço muito menor que o das próprias subcadeias. Em todos esses casos o que importa é a troca central desta seção: aceitar uma probabilidade pequena e controlável δ de erro unilateral em troca de cerca de $1{,}44\log_2(1/\delta)$ bits por elemento, independentemente do tamanho dos elementos.

6.3 Exercícios

Exercício 43. Dizemos que uma família $\mathcal{H}$ de funções hash é ***fracamente k-universal*** se para quaisquer $x_1, \ldots, x_k \in \{0,1\}^m$

$$\mathop{\mathbb{P}}_{h\in\mathcal{H}}(h(x_1) = h(x_2) = \cdots = h(x_k)) \leqslant \frac{1}{n^{k-1}}. \tag{6.13}$$

Prove que a família

$$h_{(a,b)}(x) = (ax + b \mod p) \mod 2^n$$

para $a,\ b \in \{0,1,\ldots,p-1\}$ é fracamente 2-universal se p é um primo com $m+1$ bits na sua representação binária.

Exercício 44. Seja $\mathcal{H}$ uma família qualquer de funções hash. Dados $x,\ y \in \{0,1\}^m$ e $h \in \mathcal{H}$ defina

$$\delta(x,y,h) = \begin{cases} 1, & \text{se } x \neq y \text{ e } h(x) = h(y) \\ 0, & \text{caso contrário.} \end{cases}$$

Prove que

$$\sum_{h\in\mathcal{H}} \delta(x,y,h) > \frac{|\mathcal{H}|}{2^n} - \frac{|\mathcal{H}|}{2^m}.$$

Exercício 45. Para p primo, prove que as p^{r+1} funções de $U = \{0,1,\ldots,p-1\}^{r+1}$ em $\{0,1,\ldots,p-1\}$ dadas por

$$h_{\mathsf{a}}(\mathsf{x}) = \langle \mathsf{a},\mathsf{x}\rangle \bmod p = \sum_{i=0}^{r} a_i x_i \bmod p$$

é universal (sortear uma função equivale a sortear a em U).

Exercício 46. Na seção 6.1, página 144, foi dito que se uma tabela de espalhamento usa uma função aleatória escolhida numa família universal, então o número esperado de colisões para S fixo é $\binom{n}{2}(1/m) = n(n-1)/(2m)$ de modo que se $m = n^2$ então $\mathbb{E}C < 1/2$ e não há colisão com probabilidade pelo menos $1/2$. Portanto, se S é um conjunto fixo (estático) podemos sortear uma função até que encontremos uma que não ocasiona colisões para elementos de S. Escreva um algoritmo aleatorizado que dado S encontra uma função de *hash h perfeita*. Determine a complexidade desse algoritmo. É possível usar o método das esperanças condicionais nesse caso?

Exercício 47. No filtro de Bloom em k linhas de w bits, com funções totalmente aleatórias, mostre que a probabilidade de falso positivo para $x \notin S$ é exatamente $(1-(1-1/w)^m)^k$ e compare com a cota $(m/w)^k$ da Proposição 6.3.

Exercício 48. Sejam F_1 e F_2 filtros de Bloom com as mesmas funções e o mesmo tamanho, construídos a partir de S_1 e S_2. Mostre que o OR bit a bit de F_1 e F_2 é exatamente o filtro construído a partir de $S_1 \cup S_2$. O AND bit a bit é o filtro de $S_1 \cap S_2$? Mostre que ele não tem falsos negativos para $S_1 \cap S_2$ e dê um exemplo em que tem mais bits 1 que o filtro construído diretamente a partir de $S_1 \cap S_2$.

Exercício 49. Para $m = 10^6$ e $n = 8 \times 10^6$ bits, determine o k inteiro que minimiza (6.12) e calcule o valor aproximado da probabilidade de falso positivo. Confirme, derivando duas vezes, que o ponto crítico $k = (n/m)\ln 2$ é de fato um mínimo de $g(k)$.

Exercício 50. Um fluxo tem $m = 10^9$ chegadas. Queremos que, com probabilidade pelo menos $1 - 10^{-6}$, a estimativa da frequência exceda $f(x)$ em no máximo εm, com $\varepsilon = 10^{-3}$.

1. Determine n e d pela proposição e calcule o número de contadores nd.
2. Cada contador precisa de quantos bits? Qual é o espaço total em KB?
3. Qual é o erro absoluto garantido? Para que valores de $f(x)$ a estimativa tem erro relativo de no máximo 10%?

Exercício 51. Suponha que o fluxo também possa conter remoções: remover x decrementa $C[j][h_j(x)]$ para todo j. Seja $f(y)$ a frequência líquida (chegadas menos remoções) no instante da consulta, e suponha $f(y) \geq 0$ para todo y. Seja $m' = \sum_y f(y)$.

(a) Mostre que, nesse instante, $C[j][h_j(x)] = \sum_{y\,:\,h_j(y)=h_j(x)} f(y)$.

(b) Conclua que $\widehat{f}(x) \geq f(x)$ e que $\mathbb{P}[\widehat{f}(x) > f(x) + \varepsilon m'] \leq \delta$ com os mesmos n e d.

(c) Dê um exemplo, com uma frequência líquida negativa, em que $\widehat{f}(x) < f(x)$.

(Dica: a linearidade dos contadores reduz tudo à prova da proposição, com m' no lugar de m. Para (c), uma remoção de x sem chegada anterior já serve.)

Exercício 52. Queremos um filtro para $m = 10^6$ elementos com probabilidade de falso positivo no máximo $\delta = 10^{-3}$.

1. Com k linhas e $n = 2m$, qual k basta e quantos bits o filtro usa?
2. Com um único vetor de n bits, calcule n e k pela fórmula clássica, em bits e em MB.

3. Compare com o limite inferior $m \log_2(1/\delta)$.

Exercício 53. Sejam B_1 e B_2 os filtros (com as mesmas funções $h_1, \ldots, h_k$ e o mesmo tamanho) de S_1 e S_2.

1. Mostre que o OR bit a bit $B_1 \vee B_2$ é exatamente o filtro de $S_1 \cup S_2$. Conclua que a probabilidade de falso positivo de $B_1 \vee B_2$, para $x \notin S_1 \cup S_2$, é no máximo $\left(|S_1 \cup S_2|/n\right)^k$, no filtro em k linhas.

2. Mostre, com um exemplo pequeno, que remover um elemento zerando seus k bits pode produzir um falso negativo para outro elemento de S.

3. Explique por que trocar cada bit por um contador, incrementado na inserção e decrementado na remoção, evita o problema do item (b), desde que só se removam elementos inseridos.

(Dica: em (a), cada bit de B_i vale 1 se, e somente se, algum elemento de S_i colide nele, e o filtro da união é o OR desses eventos. Para (b), basta que $a \neq b$ compartilhem uma posição.)